

Farmers Market Vendor Management System

CIS 344, Fall 2026

1. Overview

In this project, I have designed and created a Farmers market vendor management system database. To monitor markets, days the markets operate, the vendors selling on the market, what they are selling and the sales that actually occur on a particular market day. I chose this topic because it is a true type of system that is actually implemented, something I can study online with real rules written by the organization, and it has enough action to take a weak entity, a couple many-to-many relationships and a few different types of attributes..

2. Scope and Requirements Gathering

I was not able to schedule a live interview for this time, so based my questions on GrowNYC's Greenmarket program, which is an actual farmers market network in NYC, and posted its vendor rules online. That provided me with real, tangible information versus just speculation, such as vendors who need a permit that must be renewed, only able to sell certain items and vendors who apply for a market, but don't appear every day. The detailed documentation of this process, with a table of the findings converted to design decisions, is in the docs folder in requirements-gathering.docx.

3. Entities and Relationships

The system ended up with four entities:

Entity	Type	Notes
Market	Regular entity	A physical market location
MarketDay	Weak entity	Identified by Market + Date
Vendor	Regular entity	Includes permit info and category
Product	Regular entity	Owned by exactly one Vendor

And four relationships:

- HOSTS, a 1 to N identifying relationship between Market and MarketDay
- ATTENDS, a many to many relationship between Vendor and MarketDay, carrying a stall fee
- SELLS, a 1 to N relationship between Vendor and Product
- SALE, a many to many relationship between Product and MarketDay, carrying a quantity and sale price

I drew this out by hand using Chen notation first, then modeled the same design in MySQL Workbench using UML notation. Both diagrams are in the diagrams folder.

4. Database Implementation

The Chen diagram mapped fairly directly to seven tables: market, market_day, vendor, vendor_phone, product, attends, and sale. The vendor_phone table exists because Vendor has a multivalued Phone attribute, which can't be stored as a single column. The market_day table uses a composite primary key of market_id and market_date, since it's a weak entity that only makes sense in the context of its market.

I wrote three SQL scripts, in this order:

- 01_create_database.sql builds the schema and all seven tables
- 02_insert_sample_data.sql loads sample rows into every table so the database isn't empty
- 03_example_queries.sql has eight queries, including joins across the weak entity and the many to many tables, an aggregate query for total revenue per vendor, an UPDATE, and a DELETE

I tested every script against a real running MySQL database before finalizing them, not just checking that they looked right on paper.

5. Challenges

The biggest issue I ran into was MySQL Workbench crashing when I tried to place my imported tables directly onto a diagram. I worked around this by importing the tables without placing them on a diagram first, then building the diagram separately using Create Diagram from Catalog Objects, which didn't crash.

The second issue came up when I tried to use Forward Engineer to build the actual database from my model. It failed with an error saying a primary key column couldn't be NULL, even though that column was set as my primary key. This turned out to be a Workbench quirk where the NOT NULL flag didn't carry over correctly after I had reverse engineered the model from a script earlier. Instead of fighting with the model settings, I just ran my SQL script directly in a query tab connected to the live database, which worked immediately and let me confirm every table and every foreign key was created correctly.

6. Outcomes

By the end of this project I had a working MySQL database with seven tables, all the relationships from my ER diagram enforced as real foreign keys, sample data loaded in, and a set of tested queries showing the database can actually answer useful questions, like which vendor made the most revenue or which vendors have a permit expiring soon. I also have both required diagrams, a Chen version and a UML version, and they represent the same design.

7. Repository Structure

- docs/ has the mini world description and requirements gathering documentation
- diagrams/ has the hand drawn Chen diagram and the UML export from Workbench
- sql/ has the three SQL scripts, numbered in the order they should be run
- workbench/ has the .mwb file with the full Workbench model
- screenshots/ has proof the database and its relationships actually work